Implementation and Analysis of BST, AVL and RBT Data Structures

Sebastiano Babich (172249) - Leonardo Mazzon (172587)
Damiano Netto (171948)

University of Udine, Academic Year 2025 - 2026

Contents

1 Abstract

The goal of the project was to implement and compare some of the data structures studied in the "Algorithms and Data Structures" course at the University of Udine, specifically standard and self-balancing binary search trees. The project focused in particular on the time complexity involved in insertion and deletion operations within these structures. We also decided to analyze the heights reached by the trees during the various operations and to count the number of rotations performed on each tree.

2 Introduction

Search trees are among the most widely used data structures, since they enable high performance in both searching, inserting and deleting data. Specifically, the trees used in our project are binary: each node has at most two children, which can either be other nodes or null references. The resulting tree is acyclic, and one of its most important characteristics is its height that is defined as the maximum number of edges on the path from the root to any leaf.

3 Theoretical Background

In all data structures implementing search trees, height is a fundamental property: keeping its value as small as possible helps ensure efficient operations thanks to more efficient search, insertion and deletion operations.

Trees can be divided into two categories based on how they manage their height: **binary search trees** and **balanced binary trees**. Standard binary search trees do not actively control their height, while balanced binary trees use specific rules and rotation operations to keep the height as small as possible.

Trees belonging to the second category are designed to maintain a height close to the order of $\log_2(n)$, where n is the number of nodes inserted into the tree.

In this project, we studied three types of trees, two of which implement self-balancing techniques.

3.1 Binary Search Tree (BST)

Binary search trees do not impose any restrictions on their height. Their main purpose is to provide efficient search, insertion and deletion operations maintaining an ordered structure of the stored elements. However, since they do not include any mechanism to control their height, the tree can become unbalanced depending on the order in which nodes are inserted. As the number of nodes increases, this can lead to a significant degradation in performance, especially in the worst-case scenario.

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

Table 1: Time complexity in BST operations

3.2 Adelson-Velsky Landis Tree (AVL)

AVL trees are a data structure that implements self-balancing binary search trees. The search operation in the tree exactly mirrors the search in BST. Insertion initially uses the BST insertion method, with the difference that a self-balancing operation is performed afterward.

Specifically, this operation aims to identify imbalances in the tree by comparing the heights of two child nodes. Specifically, the goal is to keep total height as low as possible, so that search operations remain as close as possible to logarithmic complexity.

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 2: Time complexity in AVL operations

3.3 Red Black Tree (RBT)

...

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 3: Time complexity in RBT operations

4 Implementation

The data structures (BST, AVL and RBT) were implemented as separate modules in the project, with similar interfaces for insertion, deletion, and search. Each tree maintains additional information useful for analysis, such as current height and number of rotations. Balancing operations were separated into dedicated functions within each type of tree to comply with the rules imposed by the data structure used.
These are the main functionalities implemented for each tree:

- **BST (Binary Search Trees)**
  - **Node implementation**: a *Node* class has been implemented to store the key, the references to the left and right children, and a reference to the parent node. Each one of these could be another node, or null reference.
  - **Node insertion and deletion**: insertion and deletion operations are implemented according the standard algorithms of Binary Search Trees, maintaining the BST ordering property.
  - **Search operations**: efficient search, minimum, maximum, predecessor and successor operations are provided by exploiting the ordering property of the Binary Search Tree.
  - **Tree traversals operations**: inorder, preorder and postorder are implemented iteratively to permit the visit of each tree created.
  - **Tree rotations**: left and right rotation operations are implemented as fundamental tree transformations. These operations are reused by the balanced tree implementations and a rotation counter is kept to be used in the experimental analysis.
- **AVL (Adelson-Velsky Landis Trees)**
  - **AVLNode implementation**: an *AVLNode* class, extending the *Node* class contained in BST, has been created. Each node stores an additional *height* field, which is required to compute the balance factor.
  - **Balance factor computation**: the balance factor of each node is calculated as the difference between the heights of its left and right subtrees. That value is used to detect unbalanced configurations.
  - **Height management**: the height of each node is updated after insertions, deletions and rotations through the *update_height* function, avoiding the need of recomputing heights recursively.

- **Node insertion and deletion**: insertion and deletion operations are implemented by extending the BST algorithms and performing the required rebalancing after each update, checking the balance_factor calculated. The operations which can occour can be rotations in left or right direction.
- **Tree rotations**: rotations are used as balancing operations to restore the AVL property after insertions and deletions. These operations are partially inherited from the BST class. Single and double rotations are selected according to the imbalance configuration detected by the balance factor.

- **RBT (Red-Black Trees)**
  - **RBTNode implementation**: *RBTNode* class has been creating to extend *Node* class contained in BST. Each node in fact stores an additional *color* field, which can be either red or black, fundamental to maintain the red-black tree properties.
  - **Color management**: utility functions have been implemented to get, set and invert node colors. Missing children are considered black, simplifying the handling of boundary cases during insertion and deletion.
  - **Node insertion and deletion**: insertion and deletion are implemented by extending the standard BST functions. After each insertion or deletion, a rebalancing procedure is performed, involving recoloring operations and tree rotations. The required operations restore the red-black properties while preserving the BST ordering.
  - **Tree rotations**: left and right rotations inherited from BST are reused during the balancing procedures to modify the tree structure while preserving the ordering property.

5 Test Cases Setup

The first issue to manage during the implementation of the test cases was the choice of the input sizes on which the experiments would be performed. We chose a wide range of tree sizes to make the simulation as realistic as possible, highlighting the performance differences among the different data structures.

A valid range of values for the number of nodes had to allow the analysis in multiple orders of magnitude. Instead of using a linear distribution of input sizes, which could have caused issues in tests on large numer of elements, a geometric progression was adopted in order to obtain a more uniform distribution on a logarithmic scale.

This approach allowed us to perform a better analysis of the growth trends of the different tree implementations.

These were the parameters selected:

$$n_{min} = 1000 \qquad n_{max} = 1\,000\,000 \qquad samples = 100$$

The list of input sizes is generated through the *calc_lista_val_n* function, which computes the values of n using a geometric progression. The common ratio of the progression is defined as:

$$n_i = \lfloor n_{min} \cdot c^i \rfloor \qquad c = \left(\frac{n_{max}}{n_{min}}\right)^{\left(\frac{1}{samples-1}\right)}$$

The geometric progression produces a total of 100 samples, gradually increasing from the minimum value of nodes up to the maximum value. The first and last values of the sequence are reported below:

$$1000,\ 1072,\ 1150,\ 1233,\ 1322,\ \ldots,\ 811\,131,\ 869\,749,\ 932\,603, 1\,000\,000$$

6 Results and Discussion

The obtained results correctly reflected the theoretical concepts studied.

6.1 Insertion Tests

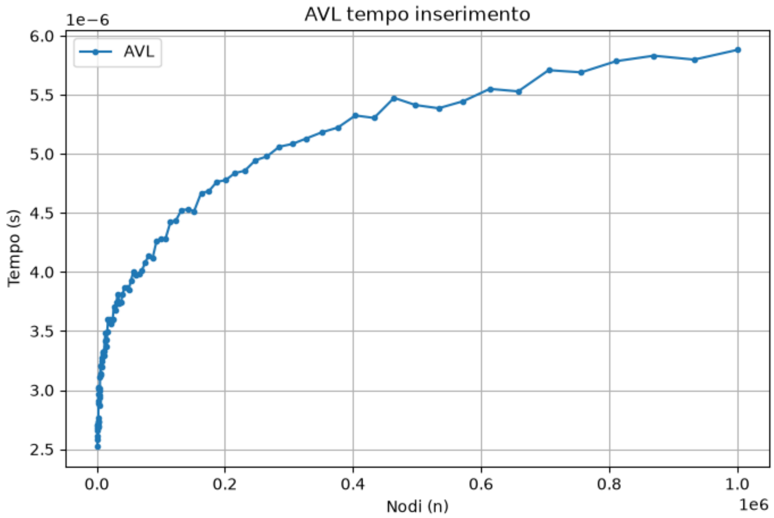

Figure 1: Test eseguito

6.2 Rotations Count Tests

6.3 Heights Tests

7 Conclusions

To conclude this project, we compared all the data structures we analyzed and drew conclusions regarding the ideal use cases for each type of tree.

7.1 Comparison of the Data Structures Analyzed

The first important distinction we noticed was between simple binary search trees and self-balancing trees:

- **Non-Balanced Binary Trees**:
- **Balanced Binary Trees**:

The other distinctions are the use cases for each data structure implemented. To drew these conclusion we analyzed the results obtained in the many tests.

- **BST**: the best use case for this type of tree are contexts where fast insertion is the primary requirement and it is necessary to avoid the computational overhead of rebalancing. In a standard BST, inserting a new element is very straightforward because it only requires traversing the tree to find the correct position, without triggering any structural rotations. This results in an excellent time complexity, on the average case, of $\mathcal{O}(\log n)$ (where n is the number of elements). However, the main drawback is that a standard BST does not guarantee balance. The worst scenario happens if the input data arrives sorted: the tree degenerates into a completely linear structure. In this case, both search and insertion performances drop significantly to $\mathcal{O}(n)$. Therefore, we would choose a BST for applications where the data is known to be random, or contexts where the dataset can be small enough that the complexity of maintaining a balanced tree is not actually necessary.

- **AVL**: for this type of balanced tree we observed a really good effort in maintaining height as low as possible of the tree, even if this results in an increased insertion time complexity. The slower insertion time, which is caused by frequent rebalancing operations, pays off significantly when searches are performed. In fact, this tree guarantees that the maximum height will never exceed $1.44 \log_2(n+2)$ (where n is the number of elements). The best-case scenario for using an AVL tree would be in systems where read operations are much more frequent than writes. In these cases we are accepting a slightly higher insertion time in exchange for really good search performance.

- **RBT**: this balanced data structure showed an excellent compromise between the two trees discussed above. Red-Black trees are particularly efficient during insertion operations, thanks to a careful and strictly limited use of rotations during the balancing procedures. Although this means the tree might not achieve the absolute minimum height, the search performance remains highly competitive, with an upper bound of $2\log_2(n+1)$ (where n is the number of elements) slightly higher than the one of AVL trees. We found many ideal use cases for this structure because it successfully combines the insertion speed of a standard BST with a search cost that is kept almost as low as an AVL. For this reason, we believe it could be a highly suitable structure to implement in systems requiring frequent searches and updates, and, more generally, in any context where maintaining a strong balance between insertion and search speed is key.

7.2 Final Remarks

The comparison demonstrates that the additional complexity required to implement self-balancing trees is justified by the significant long-term gains in efficiency and robustness offered by these data structures. While a standard BST provides a simpler implementation and execution, it remains vulnerable to worst-case scenarios keeping it less reliable for large-scale datasets. Instead, choosing between an AVL and a RBT allows us to optimize the system based on whether the specific application requires fast searches or a balanced mix of frequent updates and lookups.